

设计模式组合考题试卷一

答题建议

1. 先分析需求变化点：找出题目中哪些部分是固定的，哪些是会变化的
2. 设计类的职责：根据 "需要实现的类" 的描述，明确每个类的职责和它们之间的关系
3. 选择合适的设计模式：根据变化点和类的关系，选择 1-2 种设计模式来实现
4. 编写代码：先写接口 / 抽象类，再写实现类，最后写客户端测试代码

第 1 题：电商折扣计算系统

题目

某电商平台需要实现一个折扣计算系统，针对不同等级的用户提供不同的折扣策略：

- 普通用户：无折扣（原价）
- VIP 用户：全场 9 折
- SVIP 用户：全场 8 折，且单笔订单满 1000 元再减 100 元
- 黑钻用户：全场 7 折，且单笔订单满 500 元减 50 元，满 2000 元减 300 元

需要实现的类

1. DiscountCalculator**（折扣计算器类）**

- 功能：对外提供统一的折扣计算入口
- 包含方法：double calculateFinalPrice(String userLevel, double originalPrice)
- 输入：用户等级（normal/vip/svip/black）、订单原价
- 输出：最终支付金额

2. 各类用户的折扣计算逻辑类

- 功能：分别实现不同用户等级的折扣计算规则
- 每个类包含方法：double calculate(double originalPrice)

3. DiscountLogicProvider**（折扣逻辑提供类）**

- 功能：根据用户等级提供对应的折扣计算逻辑
- 包含方法：[折扣计算逻辑类类型] getDiscountLogic(String userLevel)

系统要求

1. 客户端只需调用DiscountCalculator的calculateFinalPrice方法即可完成计算
2. 未来可以方便地新增用户等级和对应的折扣策略，无需修改DiscountCalculator类的代码
3. 折扣策略的实现细节对客户端完全透明

```
public class Main {  
    public static void main(String[] args) {  
        DiscountCalculator dc = new DiscountCalculator(new DiscountLogicProvider());  
    }  
}
```

```

        double fp = dc.calculateFinalPrice("normal", 12.0);
        System.out.println(fp);
    }
}

class DiscountCalculator {
    private DiscountLogicProvider dlp;

    public DiscountCalculator(DiscountLogicProvider dlp) {
        this.dlp = dlp;
    }

    public double calculateFinalPrice(String userLevel, double originalPrice) {
        DiscountLogic dl = dlp.getDiscountLogic(userLevel);
        return dl.calculate(originalPrice);
    }
}

class DiscountLogicProvider {
    public DiscountLogicProvider() {
    }

    public DiscountLogic getDiscountLogic(String userLevel) {
        switch (userLevel) {
            case "normal":
                return new NormalDL();
            case "vip":
                return new VIPDL();
            case "svip":
                return new SVIPDL();
            case "black":
                return new BlackDL();
            default:
                return null;
        }
    }
}

abstract class DiscountLogic {
    public abstract double calculate(double originalPrice);
}

class NormalDL extends DiscountLogic {
    @Override
    public double calculate(double originalPrice) {
        return originalPrice;
    }
}

class VIPDL extends DiscountLogic {
    @Override
    public double calculate(double originalPrice) {
        return 0.9 * originalPrice;
    }
}

class SVIPDL extends DiscountLogic {
    @Override

```

```
    public double calculate(double originalPrice) {
        return 0.8 * originalPrice;
    }
}

class BlackDL extends DiscountLogic {
    @Override
    public double calculate(double originalPrice) {
        double price = 0.7 * originalPrice;
        if (price >= 2000) {
            return price - 300;
        } else if (price >= 500) {
            return price - 50;
        } else {
            return price;
        }
    }
}
```

第 2 题：日志记录器系统

题目

某系统需要实现一个日志记录器工厂，负责创建不同级别的日志记录器：

- 调试日志（DEBUG）：输出到控制台
- 信息日志（INFO）：输出到控制台和本地文件
- 错误日志（ERROR）：输出到控制台、本地文件和发送邮件通知管理员

需要实现的类

1. **LoggerFactory****（日志工厂类）**

- 功能：提供日志记录器的获取功能
- 包含方法：
 - static LoggerFactory getInstance(): 获取工厂实例
 - Logger getLogger(String level): 根据日志级别获取对应的日志记录器

2. **Logger****接口及各类日志记录器实现类**

- 功能：定义日志记录的统一方法，并实现不同级别的日志记录逻辑
- 包含方法：void log(String message)

系统要求

1. 整个系统中只能有一个LoggerFactory实例
2. 客户端通过工厂获取对应级别的日志记录器，调用统一的log()方法记录日志
3. 日志记录器的创建和初始化过程对客户端透明

```
public class Main {
    public static void main(String[] args) {
        LoggerFactory factory = LoggerFactory.getInstance();

        Logger debugLogger = factory.getLogger("DEBUG");
        Logger infoLogger = factory.getLogger("INFO");
    }
}
```

```

    Logger errorLogger = factory.getLogger("ERROR");

    debugLogger.log("This is a debug message.");
    infoLogger.log("This is an info message.");
    errorLogger.log("This is an error message.");

    // 未知级别时回退到 INFO
    Logger fallbackLogger = factory.getLogger("UNKNOWN");
    fallbackLogger.log("Unknown level fallback message.");
}
}

class LoggerFactory {
    private static LoggerFactory instance;

    public static LoggerFactory getInstance() {
        if (instance == null) {
            instance = new LoggerFactory();
        }
        return instance;
    }

    private LoggerFactory() {}

    public Logger getLogger(String level) {
        if (level == null) {
            return new InfoLogger();
        }

        switch (level.toUpperCase()) {
            case "DEBUG":
                return new DebugLogger();
            case "INFO":
                return new InfoLogger();
            case "ERROR":
                return new ErrorLogger();
            default:
                return new InfoLogger();
        }
    }
}

abstract class Logger {
    public abstract void log(String message);
}

class DebugLogger extends Logger {
    @Override
    public void log(String message) {
        System.out.println("[DEBUG] " + message);
    }
}

class InfoLogger extends Logger {
    @Override
    public void log(String message) {
        System.out.println("[INFO] " + message);
    }
}

```

```
    }  
}  
  
class ErrorLogger extends Logger {  
    @Override  
    public void log(String message) {  
        System.out.println("[ERROR] " + message);  
    }  
}
```

第 3 题：智能家居万能遥控器

题目

某智能家居公司需要开发一个万能遥控器，支持控制多种家电：

- 电视：开机、关机、切换频道
- 空调：开机、关机、调节温度
- 灯光：开灯、关灯、调节亮度

需要实现的类

1. **TV**/AirConditioner/Light**（家电类）**

- 功能：实现各类家电的具体功能
- 以TV为例，包含方法：void on()、void off()、void changeChannel()

2. **各类家电操作的封装类**

- 功能：将家电的每个操作封装成独立的类
- 每个类包含方法：void execute()、void undo()

3. **MacroOperation****（组合操作类）**

- 功能：将多个操作组合成一个 "一键操作"
- 包含方法：
 - void addOperation([操作封装类类型] operation)：添加子操作
 - void execute()：依次执行所有子操作
 - void undo()：逆序撤销所有子操作

4. **RemoteControl****（遥控器类）**

- 功能：管理按钮和对应的操作，支持撤销
- 包含方法：
 - void setCommand(int slot, [操作封装类类型] command)：设置按钮对应的操作
 - void pressButton(int slot)：按下按钮执行操作
 - void pressUndoButton()：按下撤销按钮

系统要求

1. 支持 "一键场景" 功能, 例如 "回家模式" 可以同时执行开灯、开空调、开电视三个操作
2. 支持撤销操作, 可以撤销上一次执行的命令
3. 未来可以方便地新增家电和对应的操作, 无需修改遥控器代码

```
import java.util.List;
import java.util.ArrayList;
import java.util.HashMap;
public class Main{
    public static void main(String[] args){
        TV tv = new TV();
        AirConditioner ac = new AirConditioner();
        Light light = new Light();

        Command tvonCommand = new TVOnCommand(tv);
        Command tvOffCommand = new TVOffCommand(tv);
        Command tvChangeChannelCommand = new TVChangeChannelCommand(tv, 1); // 默认频道1

        // 空调相关命令
        Command acOnCommand = new AirConditionerOnCommand(ac);
        Command acOffCommand = new AirConditionerOffCommand(ac);
        Command acChangeTempCommand = new AirConditionerChangeTemperatureCommand(ac, 26); // 默认26度

        // 灯光相关命令
        Command lightOnCommand = new LightOnCommand(light);
        Command lightOffCommand = new LightOffCommand(light);
        Command lightChangeBrightnessCommand = new LightChangeBrightnessCommand(light, 50); // 默认亮度50

        // 创建遥控器对象
        RemoteControl remote = new RemoteControl();

        // 将电视相关命令设置到对应 slot
        remote.setCommand(1, tvonCommand); // slot 1: 电视开机
        remote.setCommand(2, tvOffCommand); // slot 2: 电视关机
        remote.setCommand(3, tvChangeChannelCommand); // slot 3: 电视换台

        // 将空调相关命令设置到对应 slot
        remote.setCommand(4, acOnCommand); // slot 4: 空调开机
        remote.setCommand(5, acOffCommand); // slot 5: 空调关机
        remote.setCommand(6, acChangeTempCommand); // slot 6: 空调调温

        // 将灯光相关命令设置到对应 slot
        remote.setCommand(7, lightOnCommand); // slot 7: 灯光开灯
        remote.setCommand(8, lightOffCommand); // slot 8: 灯光关灯
        remote.setCommand(9, lightChangeBrightnessCommand); // slot 9: 灯光调亮度

        // 新增: 改变电视频道到 5
        Command tvChangeChannel5Command = new TVChangeChannelCommand(tv, 5);
        remote.setCommand(10, tvChangeChannel5Command); // slot 10: 电视换台到5

        // 新增: 改变空调温度到 24 度
        Command acChangeTemp24Command = new AirConditionerChangeTemperatureCommand(ac, 24);
        remote.setCommand(11, acChangeTemp24Command); // slot 11: 空调调到24度

        // 新增: 改变灯光亮度到 80
        Command lightChangeBrightness80Command = new LightChangeBrightnessCommand(light, 80);
```

```

remote.setCommand(12, lightChangeBrightness80Command); // slot 12: 灯光调到80亮度

// 新增：宏命令——键打开电视、空调、灯光
MacroOperation macroOn = new MacroOperation();
macroOn.addOperation(tvonCommand);
macroOn.addOperation(acOnCommand);
macroOn.addOperation(lightOnCommand);
remote.setCommand(13, macroOn);
// slot 13: 宏命令开所有设备

// 新增：宏命令——键关闭电视、空调、灯光
MacroOperation macroOff = new MacroOperation();
macroOff.addOperation(tvOffCommand);
macroOff.addOperation(acOffCommand);
macroOff.addOperation(lightOffCommand);
remote.setCommand(14, macroOff);

// 打印初始状态
System.out.println("--- 初始状态 ---");
printCurrentStatus(tv, ac, light);

// 单独调用某个指令示例
System.out.println("--- 单独调用指令 ---");
remote.pressButton(1); // 电视开机
remote.pressButton(3); // 电视切到频道1
remote.pressButton(4); // 空调开机
remote.pressButton(6); // 空调调到26度
remote.pressButton(7); // 灯光开灯
remote.pressButton(9); // 灯光亮度50

// 调用宏指令示例
System.out.println("--- 调用打开宏指令 ---");
remote.pressButton(13); // 一键打开所有设备
// 执行完命令后的状态
System.out.println("--- 执行命令后状态 ---");
printCurrentStatus(tv, ac, light);
System.out.println("--- 调用关闭宏指令 ---");

remote.pressButton(14); // 一键关闭所有设备

// 执行完命令后的状态
System.out.println("--- 执行命令后状态 ---");
printCurrentStatus(tv, ac, light);

// 撤销最近三次操作（宏关机、宏开机、灯光调亮度）
System.out.println("--- 执行撤销操作 ---");
remote.pressUndoButton();
remote.pressUndoButton();
remote.pressUndoButton();

// 撤销后的状态
System.out.println("--- 撤销后状态 ---");
printCurrentStatus(tv, ac, light);
}

private static void printCurrentStatus(TV tv, AirConditioner ac, Light light){
    System.out.println(

```

```

        "TV: state=" + tv.getState() + ", channel=" + tv.getCurrentChannel()
    );
    System.out.println(
        "AC: state=" + ac.getState() + ", temperature=" + ac.getCurrentTemperature()
    );
    System.out.println(
        "Light: state=" + light.getState() + ", brightness=" + light.getCurrentBrightness()
    );
    System.out.println();
}
}

```

```

class TV{
    private String State;
    private List<Integer> channelRecords = new ArrayList<>();

    public TV(){
        this.State = "off";
    }
    public String getState(){
        return State;
    }
    public void on(){
        this.State = "on";
    }
    public void off(){
        this.State = "off";
    }
    public void changeChannel(int channel){
        channelRecords.add(channel);
    }
    public void undoChange(){
        if(channelRecords.isEmpty()) return;
        int size = channelRecords.size();
        channelRecords.remove(size - 1);
    }
    public Integer getCurrentChannel(){
        if(channelRecords.isEmpty()) return 0;
        else return channelRecords.get(channelRecords.size() - 1);
    }
}

```

```

class AirConditioner{
    private String State;
    private List<Integer> temperatureRecords = new ArrayList<>();

    public AirConditioner(){
        this.State = "off";
    }
    public String getState(){
        return State;
    }
    public void on(){
        this.State = "on";
    }
    public void off(){
        this.State = "off";
    }
}

```



```

public void changeTemperature(int temp){
    temperatureRecords.add(temp);
}

public void undoChange(){
    if(temperatureRecords.isEmpty()) return;
    int size = temperatureRecords.size();
    temperatureRecords.remove(size - 1);
}

public Integer getCurrentTemperature(){
    if(temperatureRecords.isEmpty()) return 0;
    else return temperatureRecords.get(temperatureRecords.size() - 1);
}
}

```

```

class Light{
    private String State;
    private List<Integer> lightRecords = new ArrayList<>();

    public Light(){
        this.State = "off";
    }

    public String getState(){
        return State;
    }

    public void on(){
        this.State = "on";
    }

    public void off(){
        this.State = "off";
    }

    public void changeBrightness(int brightness){
        lightRecords.add(brightness);
    }

    public void undoChange(){
        if(lightRecords.isEmpty()) return;
        int size = lightRecords.size();
        lightRecords.remove(size - 1);
    }

    public Integer getCurrentBrightness(){
        if(lightRecords.isEmpty()) return 0;
        else return lightRecords.get(lightRecords.size() - 1);
    }
}

```

```

abstract class Command{
    public abstract void execute();
    public abstract void undo();
}

```

```

class TVOnCommand extends Command{
    private TV tv;
    public TVOnCommand(TV tv){
        this.tv = tv;
    }

    @Override
    public void execute(){
        tv.on();
    }
}

```

```

@Override
public void undo(){
    tv.off();
}
}

// TV 关机命令
class TVOffCommand extends Command{
    private TV tv;
    public TVOffCommand(TV tv){
        this.tv = tv;
    }
    @Override
    public void execute(){
        tv.off();
    }
    @Override
    public void undo(){
        tv.on();
    }
}

// TV 换台命令
class TVChangeChannelCommand extends Command{
    private TV tv;
    private int channel;
    public TVChangeChannelCommand(TV tv, int channel){
        this.tv = tv;
        this.channel = channel;
    }
    @Override
    public void execute(){
        tv.changeChannel(channel);
    }
    @Override
    public void undo(){
        tv.undoChange(); // 恢复到上一个频道
    }
}

// 空调开机命令
class AirConditionerOnCommand extends Command{
    private AirConditioner ac;
    public AirConditionerOnCommand(AirConditioner ac){
        this.ac = ac;
    }
    @Override
    public void execute(){
        ac.on();
    }
    @Override
    public void undo(){
        ac.off();
    }
}

// 空调关机命令
class AirConditionerOffCommand extends Command{

```

```

private AirConditioner ac;
public AirConditionerOffCommand(AirConditioner ac){
    this.ac = ac;
}
@Override
public void execute(){
    ac.off();
}
@Override
public void undo(){
    ac.on();
}
}

```

// 空调调温命令

```

class AirConditionerChangeTemperatureCommand extends Command{
    private AirConditioner ac;
    private int temperature;
    public AirConditionerChangeTemperatureCommand(AirConditioner ac, int temperature){
        this.ac = ac;
        this.temperature = temperature;
    }
    @Override
    public void execute(){
        ac.changeTemperature(temperature);
    }
    @Override
    public void undo(){
        ac.undoChange(); // 恢复到上一个温度
    }
}

```

// 灯光开灯命令

```

class LightOnCommand extends Command{
    private Light light;
    public LightOnCommand(Light light){
        this.light = light;
    }
    @Override
    public void execute(){
        light.on();
    }
    @Override
    public void undo(){
        light.off();
    }
}

```

// 灯光关灯命令

```

class LightOffCommand extends Command{
    private Light light;
    public LightOffCommand(Light light){
        this.light = light;
    }
    @Override
    public void execute(){
        light.off();
    }
}

```

```

@Override
public void undo(){
    light.on();
}
}

// 灯光调亮度命令
class LightChangeBrightnessCommand extends Command{
    private Light light;
    private int brightness;
    public LightChangeBrightnessCommand(Light light, int brightness){
        this.light = light;
        this.brightness = brightness;
    }
    @Override
    public void execute(){
        light.changeBrightness(brightness);
    }
    @Override
    public void undo(){
        light.undoChange(); // 恢复到上一个亮度
    }
}

class RemoteControl{
    private HashMap<Integer,Command> map = new HashMap<>();
    private List<Integer> record = new ArrayList<>();
    public void setCommand(int slot,Command command){
        map.put(slot, command);
    }
    public void pressButton(int slot){
        if(!map.containsKey(slot)) return;
        Command tCommand = map.get(slot);
        tCommand.execute();
        record.add(slot);
    }
    public void pressUndoButton(){
        if (record.isEmpty()) {
            return ;
        }
        int last = record.get(record.size() - 1);
        Command tCommand = map.get(last);
        tCommand.undo();
        record.remove(record.size() - 1);
    }
}

class MacroOperation extends Command{
    List<Command> commandList = new ArrayList<>() ;
    public void addOperation(Command operation){
        commandList.add(operation);
    }
    @Override
    public void execute(){
        commandList.forEach(command->command.execute());
    }
    @Override
    public void undo(){

```

```
int size = commandList.size();
for(int i = size - 1; i >= 0; i--){
    commandList.get(i).undo();
}
}
```

第 4 题：新闻推送系统

题目

某新闻网站需要实现一个新闻推送系统：

1. 当有新的新闻发布时，所有订阅了该新闻类别的用户都会收到推送通知
2. 用户可以随时订阅或取消订阅某个新闻类别
3. 整个系统只有一个新闻发布中心，所有新闻都通过这个中心发布
4. 支持多种推送方式：短信推送、邮件推送、APP 推送

需要实现的类

1. NewsPublisher（新闻发布中心类）

- 功能：管理用户订阅、发布新闻、推送通知
- 包含方法：
 - static NewsPublisher getInstance(): 获取发布中心实例
 - void subscribe(String category, [用户类类型] user): 用户订阅新闻类别
 - void unsubscribe(String category, [用户类类型] user): 用户取消订阅
 - void publishNews(String category, String content): 发布新闻并推送给订阅用户

2. User**接口及各类推送方式的用户实现类**

- 功能：定义接收新闻通知的统一方法，并实现不同推送方式的逻辑
- 包含方法：void receiveNews(String category, String content)

系统要求

1. 整个系统只有一个NewsPublisher实例
2. 用户可以随时订阅或取消订阅，新闻发布后实时推送给所有订阅用户
3. 支持多种推送方式，未来可以方便地新增推送方式

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        NewsPublisher newsPublisher = NewsPublisher.getInstance();
        User user1 = new User(new EmailStrategy());
        newsPublisher.subscribe("sports", user1);
        newsPublisher.subscribe("policy", new User(new SmsStrategy()));
        newsPublisher.subscribe("economy", new User(new AppStrategy()));
        newsPublisher.publishNews("sports", "James win");
    }
}
```

```

        newsPublisher.publishNews("policy", "china policy");
        newsPublisher.publishNews("economy", "us economy");
        newsPublisher.unsubscribe("sports", user1);
        newsPublisher.publishNews("sports", "james win");
    }
}

class NewsPublisher{
    private HashMap<String, User> users = new HashMap<>();
    private static NewsPublisher instance;
    private NewsPublisher(){

    }
    public static NewsPublisher getInstance(){
        if(instance == null){
            instance = new NewsPublisher();
        }
        return instance;
    }
    public void subscribe(String category, User user){
        users.put(category, user);
    }
    public void unsubscribe(String category, User user){
        users.remove(category, user);
    }
    public void publishNews(String category, String news){
        for(String c : users.keySet()){
            if(c.equals(category)){
                users.get(category).receiveNews(category, news);
            }
        }
    }
}

class User{
    private Strategy strategy;
    public User(Strategy strategy){
        this.strategy = strategy;
    }
    public void receiveNews(String category, String news){
        strategy.publishNews(category, news);
    }
}

abstract class Strategy{
    public abstract void publishNews(String category, String news);
}

class EmailStrategy extends Strategy{
    @Override
    public void publishNews(String category, String news) {
        System.out.println("emailStrategy publishNews category:"+category+" news:"+news);
    }
}

class SmsStrategy extends Strategy{
    @Override
    public void publishNews(String category, String news) {
        System.out.println("smsStrategy publishNews category:"+category+" news:"+news);
    }
}

```

```
class AppStrategy extends Strategy{
    @Override
    public void publishNews(String category,String news) {
        System.out.println("appStrategy publishNews category:"+category+" news:"+news);
    }
}
```

第 5 题：电脑组装系统

题目

某电脑厂商需要实现一个电脑组装系统，生产三种类型的电脑：

- 办公电脑：CPU i3，内存 8GB，硬盘 512GB SSD，集成显卡
- 游戏电脑：CPU i7，内存 16GB，硬盘 1TB SSD，独立显卡 RTX4060
- 设计电脑：CPU i9，内存 32GB，硬盘 2TB SSD，专业显卡 RTX4090

需要实现的类

1. **Computer****（电脑类）**

- 功能：表示一台完整的电脑，包含 CPU、内存、硬盘、显卡等属性
- 包含各属性的 getter 和 setter 方法，以及toString()方法

2. 各类电脑的组装类

- 功能：负责组装不同类型电脑的各个部件
- 每个类包含方法：
 - void buildCpu(): 安装 CPU
 - void buildMemory(): 安装内存
 - void buildHardDisk(): 安装硬盘
 - void buildGraphicsCard(): 安装显卡
- Computer getComputer(): 获取组装好的电脑

3. **ComputerAssembler****（电脑组装工厂类）**

- 功能：根据电脑类型调用对应的组装类完成组装
- 包含方法：static Computer assembleComputer(String type)
 - 输入：电脑类型（office/game/design）
 - 输出：组装好的完整电脑

系统要求

1. 电脑的组装过程是固定的：安装 CPU→安装内存→安装硬盘→安装显卡
2. 客户端只需指定电脑类型，即可得到组装好的完整电脑
3. 未来可以方便地新增电脑类型，无需修改现有组装逻辑

```

class ComputerAssembler{
    private static Builder builder = new ConcreteBuilder();

    public static Computer assembleComputer(String type) {
        switch (type) {
            case "office":
                builder.buildCpu("i3");
                builder.buildMemory("8GB");
                builder.buildHardDisk("512GB");
                builder.buildGraphicsCard("集成显卡");
                break;
            case "gaming":
                builder.buildCpu("i7");
                builder.buildMemory("16GB");
                builder.buildHardDisk("1TB");
                builder.buildGraphicsCard("RTX4060");
                break;
            case "design":
                builder.buildCpu("i9");
                builder.buildMemory("32GB");
                builder.buildHardDisk("2TB");
                builder.buildGraphicsCard("RTX4090");
                break;
            default:
                break;
        }
        return builder.getComputer();
    }
}

abstract class Builder{
    abstract void buildCpu(String cpu);
    abstract void buildMemory(String memory);
    abstract void buildHardDisk(String hardDisk);
    abstract void buildGraphicsCard(String graphicsCard);
    abstract Computer getComputer();
}

class ConcreteBuilder extends Builder{
    private Computer computer = new Computer();
    @Override
    public void buildCpu(String cpu) {
        computer.cpu = cpu;
    }
    @Override
    public void buildMemory(String memory) {
        computer.memory = memory;
    }
    @Override
    public void buildHardDisk(String hardDisk) {
        computer.hardDisk = hardDisk;
    }
    @Override
    public void buildGraphicsCard(String graphicsCard) {
        computer.graphicsCard = graphicsCard;
    }
    @Override
    public Computer getComputer() {
        return computer;
    }
}

```



```

    }

}

class Computer{
    String cpu;
    String memory;
    String hardDisk;
    String graphicsCard;
    public String getCpu() {
        return cpu;
    }
    public String getMemory() {
        return memory;
    }
    public String getHardDisk() {
        return hardDisk;
    }
    public String getGraphicsCard() {
        return graphicsCard;
    }
    public String toString() {
        return "Computer{" +
            "cpu='" + cpu + '\'' +
            ", memory='" + memory + '\'' +
            ", hardDisk='" + hardDisk + '\'' +
            ", graphicsCard='" + graphicsCard + '\'' +
            '}';
    }
}

public class Main {

    public static void main(String[] args) {
        Computer office = ComputerAssembler.assembleComputer("office");
        System.out.println(office.toString());
        Computer gaming = ComputerAssembler.assembleComputer("gaming");
        System.out.println(gaming.toString());
        Computer design = ComputerAssembler.assembleComputer("design");
        System.out.println(design.toString());
    }

}

```

[java编辑器](#)

